Coding Agents as a Mechanism for Formalizing and Transferring Domain Knowledge in DNA Origami Design

Daniel Fu, Yonggang Ke

Abstract

This work investigates the use of a coding agent to design and manipulate DNA origami structures through the caDNAno2 design interface. The design methodology produced by the agent is saved as transferable, repeatable text-file instructions interpretable by another coding agent, converting informal domain expertise into formalized, distributable protocols. We demonstrate this through a parametric pipeline for simple single and multi-layer designs, identifying several failure modes that each required explicit domain knowledge from the human designer. Once encoded, this knowledge enabled the agent to autonomously extend designs to arbitrary lattice dimensions, generate structures to physical specifications, execute targeted edits from natural language prompts, and run the full pipeline from design through molecular dynamics simulation.

1. Introduction

Modifying a DNA origami design requires re-routing the scaffold, re-placing staples, and re-verifying the structure for each parameter change[1,2]. A single redesign of a simple structure may be quick, taking several minutes, but multiple iterations or greater complexity accumulate to hours of repetitive, error-prone work. The knowledge required to correctly perform these tasks exists largely as informal expertise: conventions transmitted between designers, acquired through trial and error, and rarely documented in systematic or machine-readable form.

The effort to algorithmically accelerate the design process and eliminate design tedium has been approached from various angles. A substantial ecosystem of automated design tools now exists, spanning lattice-based design[3], automated staple breaking[2], curved and tubular structures[4], wireframe origami from 2D and 3D meshes[5–8], and interactive 3D editing environments[9]. However, there is little continuity between these tools. Each addresses a specific shape class or design subtask, embodying its own bespoke design principles and operating through a disconnected interface. When a new design principle or structural modality emerges, it typically requires building another proprietary tool from the ground up. If these methods were instead consolidated into a single accessible repository, could a coding agent serve as a unified front end, learning from all prior methods and applying the appropriate design logic to each new target? In this work, we initiate that investigation by retracing the DNA origami design chronology[10–14], from the beginning; folding simple single and multi-layer designs, and developing the methodology for scaling toward greater structural complexity.

A coding agent is a large language model (LLM) that operates by reading files, writing and executing code, and iterating on the results within a command-line environment. Unlike chatbot-style LLMs that produce only text responses, coding agents take actions: they can navigate a codebase, call APIs, run scripts, inspect outputs, and modify files across multiple steps without human intervention between steps. Recent advances in LLM reasoning and tool use have made these agents capable of sustained, multi-step

software engineering tasks[15,16], including navigating unfamiliar codebases, integrating independently developed software packages, and writing domain-specific scripts from documentation and source code alone. However, LLMs have no geometric understanding. They cannot reason about nucleotide positions in 3D space. caDNAno provides a layer of abstraction that makes this unnecessary. It encodes helix connectivity, crossover placement, and strand routing as structured data rather than spatial coordinates, reducing DNA origami design to operations on lists and indices. This abstraction makes caDNAno a tractable conduit for investigating whether a coding agent can acquire the domain knowledge needed to manipulate DNA origami designs through direct interaction with the software, rather than with the 3D structure itself.

In the current work, we demonstrate how a coding agent with direct access to caDNAno's source code can read internal data models, write and execute Python scripts, and integrate external tools, like tacoxDNA[17] and oxDNA[18–21], into an end-to-end pipeline. Critically, the methodology that emerges from this process is not ephemeral. It is saved as verifier functions, parametric scripts, and documented failure catalogs: transferable, repeatable text-file instructions interpretable by another coding agent. This converts *ad hoc* design tasks into repeatable, iterable procedures and provides a mechanism for formalizing design principles that have previously lacked systematic documentation.

As a proof of concept, we demonstrate the construction of a parametric pipeline for a 2-layer rectangular origami with a tunable cavity using a coding agent (Claude Code). Achieving functional usage required iterating through 10 distinct failure modes, each corresponding to domain knowledge invisible in the source code (e.g., honeycomb grid-to-layer mapping, crossover parity rules, cavity orientation conventions). Each failure was resolved by explicit correction from the human designer, after which the agent did not repeat that error class. Once this knowledge was encoded, the agent autonomously extended the base template to arbitrary lattice dimensions, generated designs to physical specifications, executed targeted structural edits from natural language prompts, and ran the full pipeline from design through stapling to oxDNA molecular dynamics simulation.

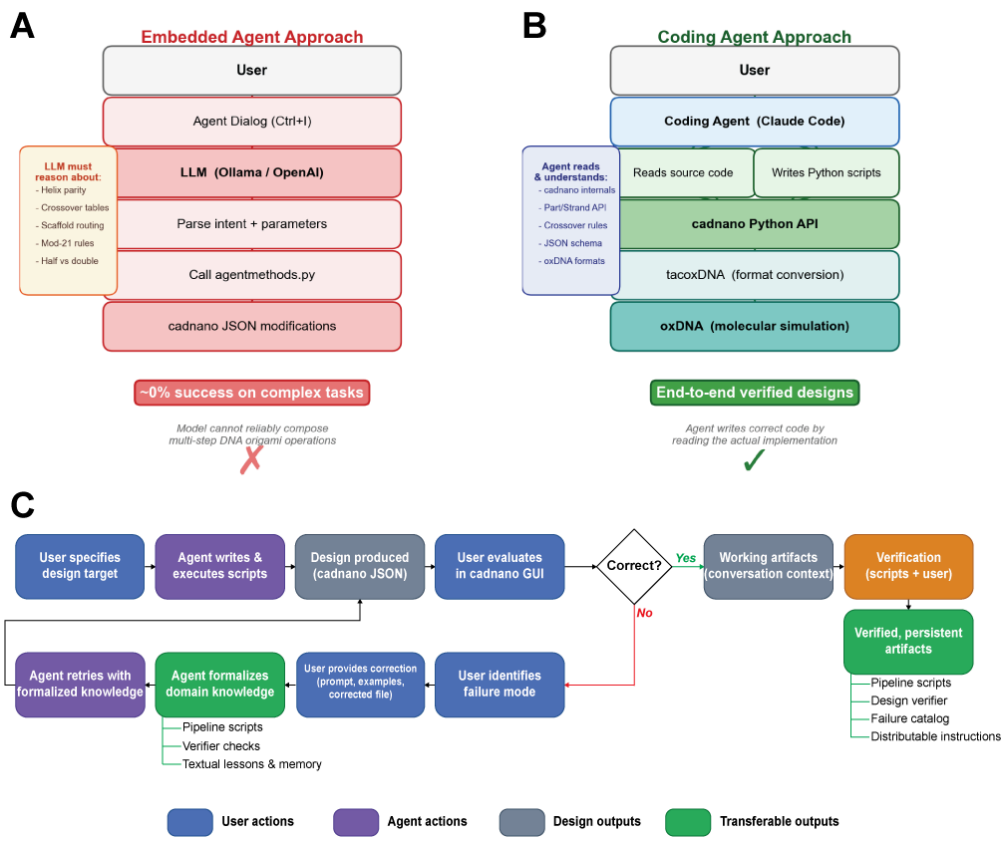

Figure 1: Architecture comparison. (A) Embedded tool-using LLM achieved 0% success on multi-step design tasks because the model could not resolve contextual constraints (helix parity, crossover directionality) from tool schemas alone. (B) Coding agent with source code access produced end-to-end verified designs by reading caDNAno internals, writing Python scripts, and integrating external simulation tools. (C) Knowledge formalization workflow. The user specifies a design target; the agent writes and executes scripts; the user evaluates the result. On failure, the user identifies the failure mode and provides a correction, which the agent formalizes as pipeline scripts, verifier checks, and persistent lessons. The agent retries with this formalized knowledge until the design is correct. On success, working artifacts pass through verification to become persistent, distributable outputs: parametric pipelines, an automated design verifier, a failure catalog, and text-file instructions interpretable by any agent.

2. Tool-Calling LLM: Insufficient Domain Knowledge from Schemas

Before describing the established pipeline, we distinguish two architectures for applying an LLM to a software tool: tool-calling, where the model selects from predefined functions, and code generation, where the model reads source code and writes scripts. This section and the next describe our experience with each. The distinction determines what kinds of domain knowledge the model can acquire and apply.

Our initial approach was to embed an LLM directly into the caDNAno GUI as a tool-calling agent (Figure 1, A): the user types a natural language command, the model parses the intent, and calls predefined Python methods that modify the design. The premise was that if we could decompose DNA origami manipulation into a sufficient set of primitives (identify crossovers, move them, place strands, break staples), the model would be able to compose these primitives to execute arbitrary design instructions.

In practice, each primitive required contextual parameters that the model could not resolve from the tool schema. A "place crossover" function must distinguish half-crossovers from full crossovers, left-edge from right-edge from interior positions, and $5' \rightarrow 3'$ from $3' \rightarrow 5'$ scaffold direction on the target helix. Attempts to encode these distinctions into the API surface produced a combinatorial expansion of tool variants without improving task success. Multi-step instructions such as scaffold routing (placing crossovers to form one continuous scaffold loop) achieved 0% success across GPT-class models. The model could invoke individual tools but could not satisfy the inter-step constraints that govern valid compositions. Reinforcement learning with a design verifier as reward signal[22] also failed: a local model (Qwen 1.7B) could not discover correct action sequences through exploration, which is unsurprising given that substantially more capable GPT-class models achieved 0% success on the same tasks. The GPT-class failure also precluded distillation, as no successful trajectories were available to serve as training data for the local model. Human-demonstrated trajectories were not evaluated as a data source; the volume of demonstrations required to cover the constraint space may be intractable, though we did not confirm this empirically.

The limitation was architectural. A predefined tool API encodes necessary but insufficient domain knowledge: the model can see the available actions but not the reasoning that governs when and how to compose them. Wang et al. demonstrated that tool-calling architectures are systematically inferior to code-generating agents for complex multi-step tasks, precisely because code provides arbitrary composition through control flow, state management, and dynamic parameter construction, capabilities that fixed function schemas cannot express[23]. Where a tool-calling agent must select from a predetermined menu of operations, a coding agent can read source code to understand *why* an operation exists, discover undocumented methods, and compose novel sequences that no API designer anticipated.

3. Code-Generating Agent: Domain Knowledge from Source Code

We replaced the tool-calling architecture with a coding agent (Claude Code) that reads caDNAno's source code directly and writes Python scripts (Figure 1, B). Source code access provided two capabilities absent from the tool-calling approach:

- **Discovery through code reading.** The agent identified internal methods not exposed through any public API by reading the strand model source. For example, when the public `createXover` method broke scaffold continuity by splitting strands, the agent located `setConnection3p/5p` in the strand model internals, which preserves connectivity.
- **End-to-end pipeline construction.** The agent integrated caDNAno with tacoxDNA (format conversion) and oxDNA (molecular dynamics) into a single automated pipeline. Each tool is open-source and individually straightforward, but manually shepherding a design through the full sequence (export, convert, configure simulation parameters, submit, parse results) is tedious and error-prone. The agent automated this by reading each tool's documentation and source code, resolving format requirements, and writing wrapper scripts.

However, source code access alone was not sufficient. Building a correct DNA origami design required domain knowledge that is not encoded in the codebase, such as lattice geometry conventions, crossover placement rules, and scaffold routing patterns. Acquiring this knowledge required iterative human correction. The following sections

describe three categories of domain knowledge the agent could not derive from code, the process by which each was resolved, and the artifacts that formalized each correction.

4. Formalizing Helix Placement on the Honeycomb Lattice

The agent's earliest outputs were structurally incoherent (Figure 2, A). Helices were placed at arbitrary grid positions, crossovers were sparse or absent, there were many short and disconnected strands, primarily due to not understanding polarity within the design space, and the resulting designs bore little resemblance to valid DNA origami. These failures established the baseline: the agent could read caDNAno's source code and execute its API, but had no understanding of the geometric conventions that govern helix placement on the honeycomb lattice.

4.1 Autonomous Geometric Verification via tacoxDNA

One capability the agent developed independently was geometric verification through the end-to-end pipeline. caDNAno's 2D path view provides no spatial information about the 3D conformation of a design. However, with tacoxDNA integrated, the agent could convert candidate designs to 3D coordinates and analyze the resulting geometry. The agent used this to determine whether a helix arrangement was physically "flat": it generated candidate grid layouts, converted each to oxDNA format, computed cross-sectional slices by PCA decomposition of the 3D nucleotide coordinates, and evaluated whether the resulting profile matched a flat rectangle (Figure 2, B). By measuring cross-sectional circularity (flat sheet: aspect ratio ≈ 0.05; incorrect 2-by-3 grid: ≈ 0.84), the agent could reject wrong helix arrangements without human feedback. This process, integration of a 3D simulation tool to compensate for geometric information absent from the design tool, was not anticipated in the original system design.

4.2 The "Layer" Ambiguity

Despite this geometric capability, the agent could not resolve the relationship between grid rows and physical layers on the honeycomb lattice. On the honeycomb lattice, helices are arranged in a hexagonal packing where adjacent rows are offset vertically. A single row of hexagonally packed helices occupies two distinct y-coordinates, because the vertices of a hexagon do not lie on a single horizontal line. The agent interpreted these two y-coordinates as two physical layers, and therefore constructed a 4-by-7 grid (4 rows, 7 columns) when instructed to build a "2-layer" rectangle. In fact, a 2-layer structure corresponds to 2 grid rows on the honeycomb lattice (yielding a 2-by-12 grid), because one physical layer comprises helices at both y-coordinates of a single hexagonal row.

This ambiguity could not be resolved from the source code. The agent independently generated a multiple-choice diagram of 1-layer, 2-layer, 3-layer, and 4-layer cross-sections and asked the user to identify which arrangement corresponded to "2-layer" (Figure 2, C). The user labeled the correct option, and the agent did not repeat this error in any subsequent session. The correction was formalized as a constraint in the pipeline: N-layer = N grid rows on the honeycomb lattice.

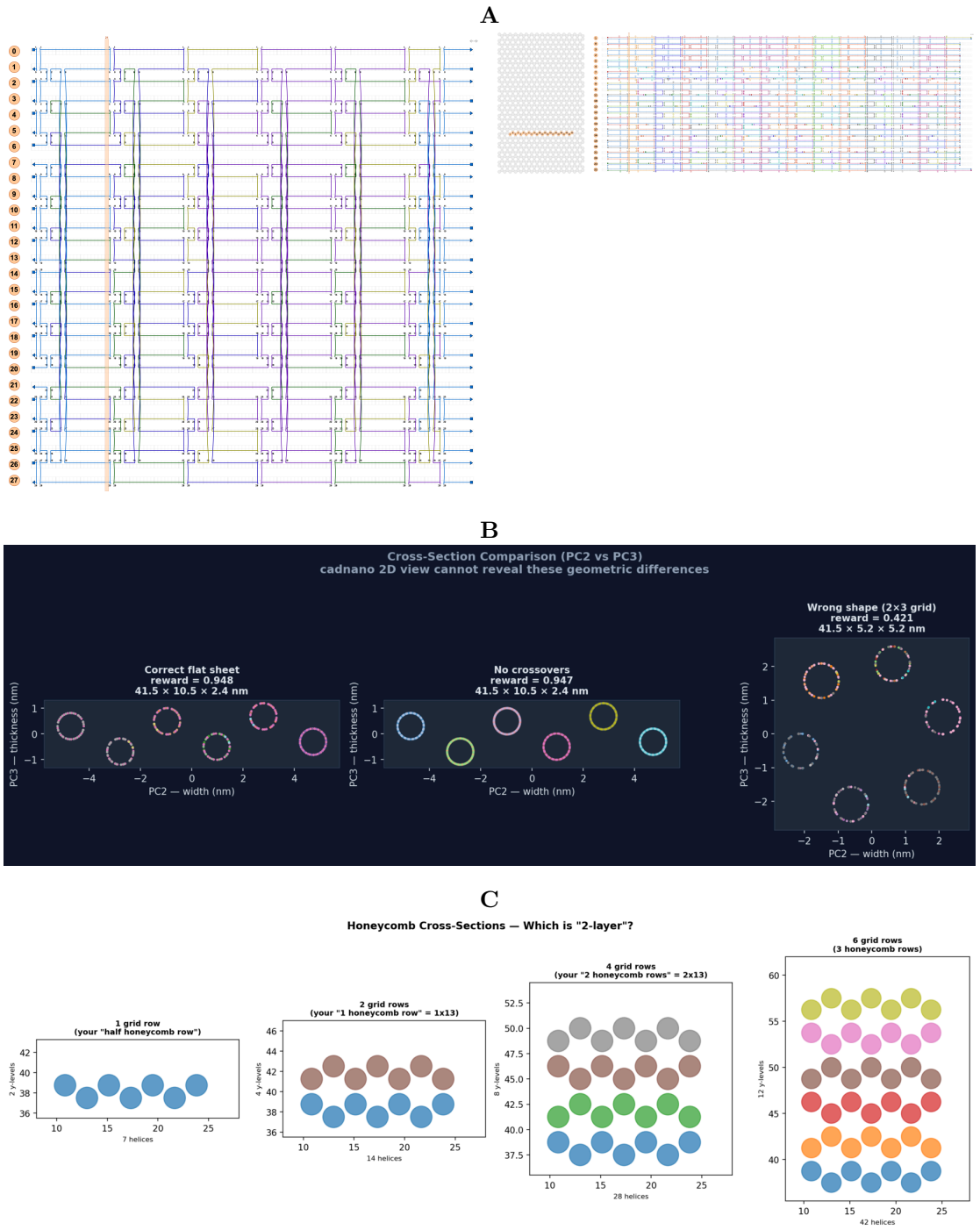

Figure 2: Formalizing helix placement on the honeycomb lattice. (A) Early agent outputs: a 4-by-7 grid (left, the agent's incorrect interpretation of "2-layer") and a flat rectangle with serpentine routing (right, structurally valid but with incorrect scaffold routing). (B) Autonomous geometric verification via tacoxDNA: PCA cross-section analysis distinguishes correct flat sheets (reward 0.948) from incorrect grid arrangements (reward 0.421). The agent developed this verification independently by converting caDNAno designs to oxDNA 3D coordinates and measuring cross-sectional circularity. (C) Multiple-choice diagram the agent generated to resolve the "layer" ambiguity. The user identified the correct grid-to-layer correspondence, which the agent retained for all subsequent designs.

5. Formalizing Scaffold Routing Rules

With lattice geometry resolved, the next class of errors concerned scaffold routing: how crossovers connect the scaffold strand into a single continuous loop across all helices.

5.1 Serpentine vs Dense Routing

The agent's initial approach was serpentine routing: one crossover per helix pair, producing 65 disconnected scaffold oligos instead of a single continuous strand (Figure 3, A). Correct DNA origami routing requires dense crossover placement: the scaffold weaves between adjacent helices through multiple double crossovers per pair, with half-crossovers at the left and right edges of the structure serving as turn points (Figure 3, B). The agent had no basis for distinguishing which half-crossovers belong on which edge, or how full crossovers and half-crossovers must alternate to produce a single continuous scaffold path.

5.2 User Correction

The user provided a hand-designed 2-by-12 template with correct dense crossover routing (Figure 3, C), along with an explanation of half-crossover versus full-crossover placement rules. The key distinction: at each helix pair boundary, the scaffold turns via a single half-crossover (left edge or right edge, determined by helix parity), while the interior is connected by double crossovers at every valid honeycomb position. This "single midseam" pattern generalizes to all simple rectangular structures.

Additional errors in this phase included scaffold/staple strand assignment (the agent placed scaffold strands on the staple strand set and vice versa), which the user identified by inspecting the design in caDNAno.

After these corrections, the agent produced a valid 2-by-14 2-layer honeycomb DNA origami rectangle, which served as the first end-to-end validation of the pipeline from design through molecular dynamics simulation.

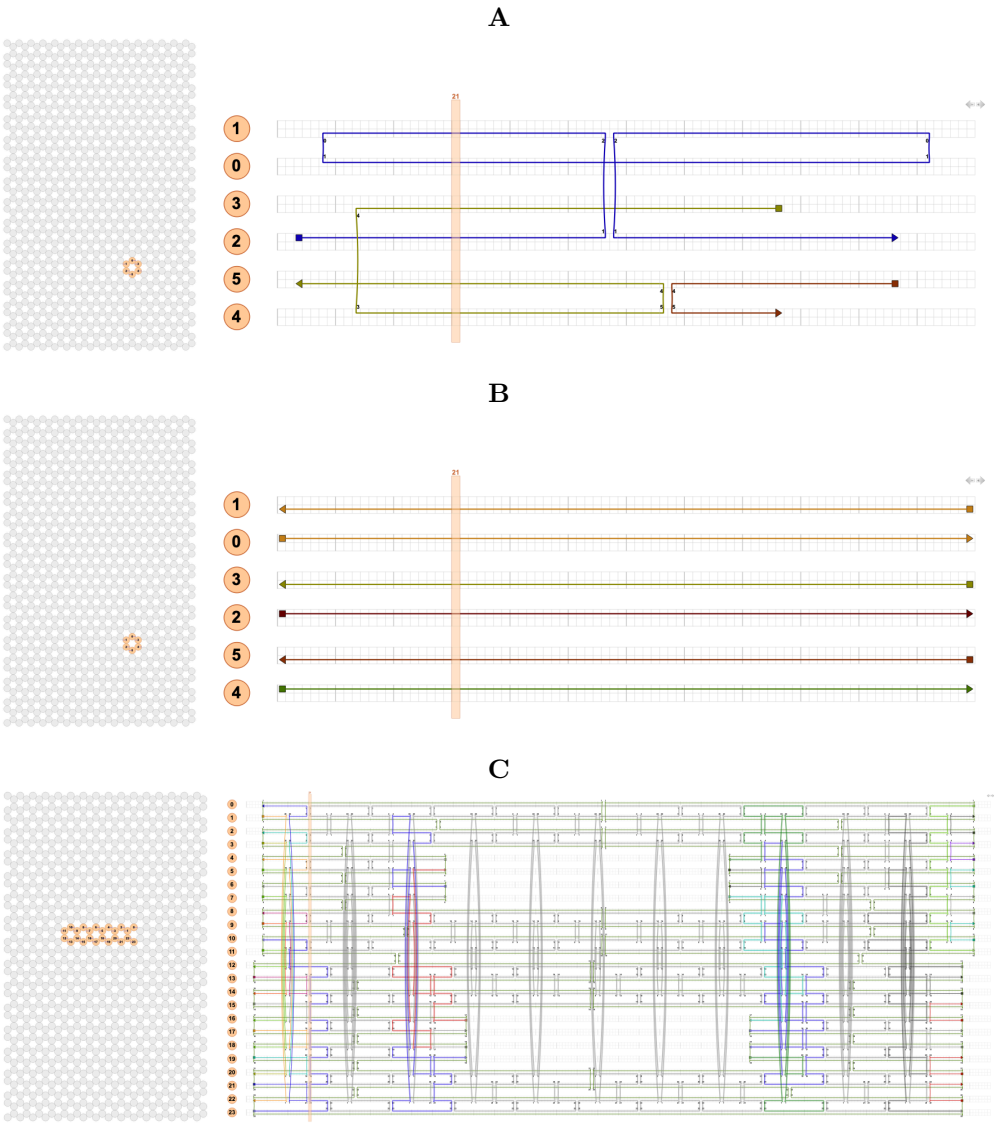

Figure 3: Formalizing scaffold routing. (A) Agent's from-scratch attempt produced 65 scaffold oligos using serpentine routing (1 crossover per helix pair). (B) Disconnected helices with no crossovers (6 scaffold oligos). (C) User-provided template with correct dense crossover routing, 1 scaffold oligo (5,036 bp). The half-crossover vs full-crossover distinction and the "single midseam" routing pattern were explained alongside this template.

6. Formalizing Cavity Design

With scaffold routing established for simple rectangles, we introduced a more complex design goal: a 2-layer rectangle with a rectangular cavity, a gap in the interior of the structure along the helix length axis. This design used the p8064 scaffold (8,064 nt) and introduced a second set of failure modes.

6.1 Cavity Failures

The agent placed the cavity by removing helices from the XY cross-section (altering the grid layout), when the cavity should instead be a gap along the helix length axis (Z direction). The user corrected this by specifying the axis along which the cavity operates.

When scaling the template from 252 to 420 bp, the agent's script left 5,036 stray staple entries with broken crossover references, causing caDNAno to crash on load. The user traced the crash to `legacydecoder.py` line 205 and identified the stray references as the cause. In a separate instance, the agent filled the cavity gap with continuous scaffold when extending the design, destroying the cavity; the user manually cleared the affected positions and returned the corrected file.

The agent's scaling logic also moved crossover positions independently per helix, creating skewed crossovers (e.g., H16[337] vs H17[338]) where both sides must share the same base pair index. The user provided the corrected alignment. A related issue arose with cavity edge alignment between layers: the cavity right edge was at position 270 in one layer and 338 in the other. The nearest valid honeycomb position on each layer differs by 2 bp (340 vs 338) due to different crossover direction offsets. In this case, the agent resolved the issue autonomously by computing valid crossover positions for each honeycomb direction and selecting the closest match, which the user confirmed.

6.2 Correction and End-to-End Validation

As with lattice geometry and scaffold routing, each correction was provided through descriptive prompts or corrected design files. Once corrected, the agent retained the domain knowledge and did not repeat the error class. The agent developed a 4-step scaling process, validated at each step:

1. Remove staples (keep scaffold only). Verified loads.
2. Extend helix arrays and shift right-side crossovers. Verified loads.
3. Move midseam crossovers to center. Verified loads.
4. Expand cavity by moving boundary crossovers. Verified loads.

The resulting design was a 2-by-12 rectangular origami with an aligned cavity, 1 continuous scaffold loop (7,688 bp), scaled from the user's 252 bp template to 420 bp helices (Figure 4, C). The agent then ran the full pipeline autonomously: autoStaple (222 staples) → autoBreak → tacoxDNA (15,656 nt) → oxDNA PACE GPU relaxation (Figure 4, D).

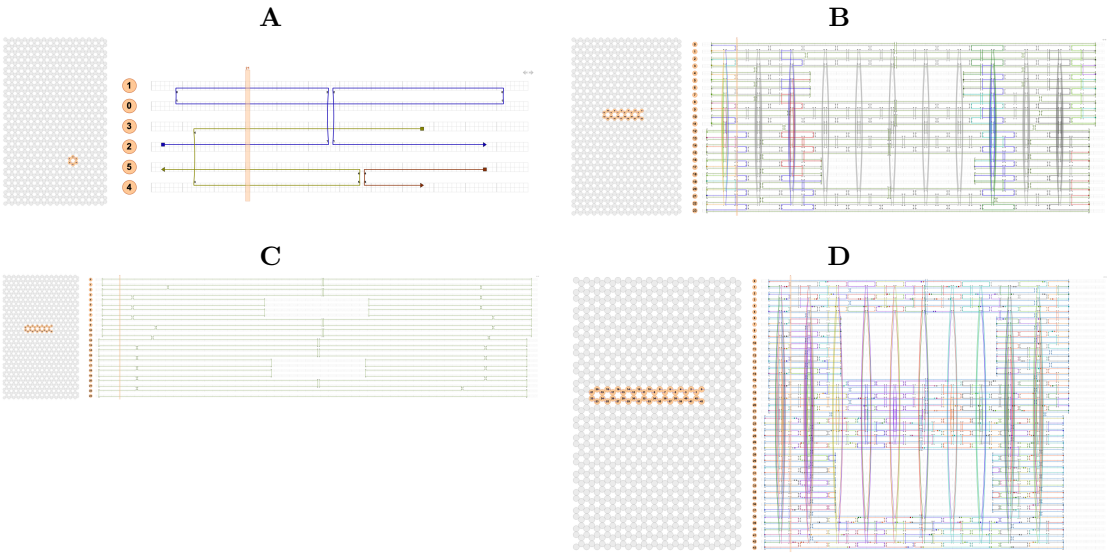

Figure 4: Formalizing cavity design. (A) Agent's initial attempt at scaffold routing: 65 disconnected oligos. (B) User-provided template with correct dense routing and cavity gap, 1 scaffold oligo (5,036 bp). (C) Correctly routed 30 nm cavity design with 1 scaffold oligo, produced by the agent after incorporating the user's corrections. The "half-and-half" routing pattern is visible: half-crossovers at cavity boundaries for scaffold turns, full crossovers in the interior. (D) Final stapled product after autoStaple + autoBreak (220+ staples, minLegLen=3), ready for tacoxDNA conversion and oxDNA molecular dynamics simulation.

7. Cumulative Capability and Targeted Edits

Each correction was retained across subsequent sessions; the agent did not repeat any resolved error class. The cumulative effect on agent capability was as follows:

1. **After lattice geometry correction** (Section 4): All subsequent designs used correct 2-by-N grid dimensions.
2. **After scaffold routing correction** (Section 5): The agent adopted the user's template as the basis for all designs, replacing from-scratch construction.
3. **After cavity orientation correction** (Section 6): All cavity designs placed the gap along the helix length axis.
4. **After scaling corrections** (Section 6): The agent developed a validated 4-step scaling process.

After correction of all 10 failure modes, the agent operated autonomously on the following tasks without further human intervention:

1. **Extended the template to arbitrary lattice dimensions** (2-by-16, 2-by-18, 2-by-20, 2-by-22) by adding column pairs to both sides of the base template, maintaining 1 scaffold oligo at each step.
2. **Designed to specification:** Given the constraint "20 nm × 40 nm cavity, centered, 6-helix padding, scaffold ≤ 8,064 bp," the agent independently selected a 2-by-20 grid (40 helices, 252 bp), computed the cavity dimensions (8 columns, 117 bp gap), and built the complete design: 7,828 bp scaffold, 1 oligo.

One-Shot Targeted Edits

To evaluate retention of transferred design knowledge, we issued three successive modification requests on a 2-by-22 integrin cavity design. The agent executed each

correctly on the first attempt without re-instruction on any previously corrected error class (Figure 5):

Edit 1: "Shrink the structure." The 2-by-22 design was wider than needed. Rather than truncating helix arrays (which would destroy edge crossovers), the agent recognized the correct approach: **move the edge crossovers inward** by one step (from positions 246/242 to 225/221), then clear scaffold data beyond the new edges. This preserved all crossover connectivity while reducing the effective helix length from 252 bp to 225 bp. Result: 7,412 bp scaffold, 1 oligo.

Edit 2: "Move the full crossovers away from the cavity edge, and center the cavity." The agent identified 5 inter-pair crossovers that were only 3–5 bp from cavity boundary half-crossovers. It moved each to a safe lattice position 26+ bp from the cavity. Then it recomputed centered cavity boundaries. Result: 7,432 bp scaffold, 1 oligo, cavity centered to within 1 bp (Figure 5, A).

Edit 3: "Now autoStaple and autoBreak it." The agent ran autoStaple (82 initial staples) and autoBreak with minStapleLegLen=3 (220+ broken staples, 18–279 bp range, preserving full crossovers). Result: complete stapled design ready for tacoxDNA conversion.

Each edit required the agent to apply multiple pieces of domain knowledge transferred in previous sessions: crossover lattice positions, parity-dependent scaffold direction, cavity boundary half-crossover placement, and honeycomb valid offsets. None of these were re-taught. The agent applied rules encoded in `tasks/lessons.md`, verifier checks in `cadnano_verifier.py`, and pipeline functions in `cavity_variant_sweep.py`. The corrections from prior sessions accumulated as reusable code that the agent applied automatically in new design contexts.

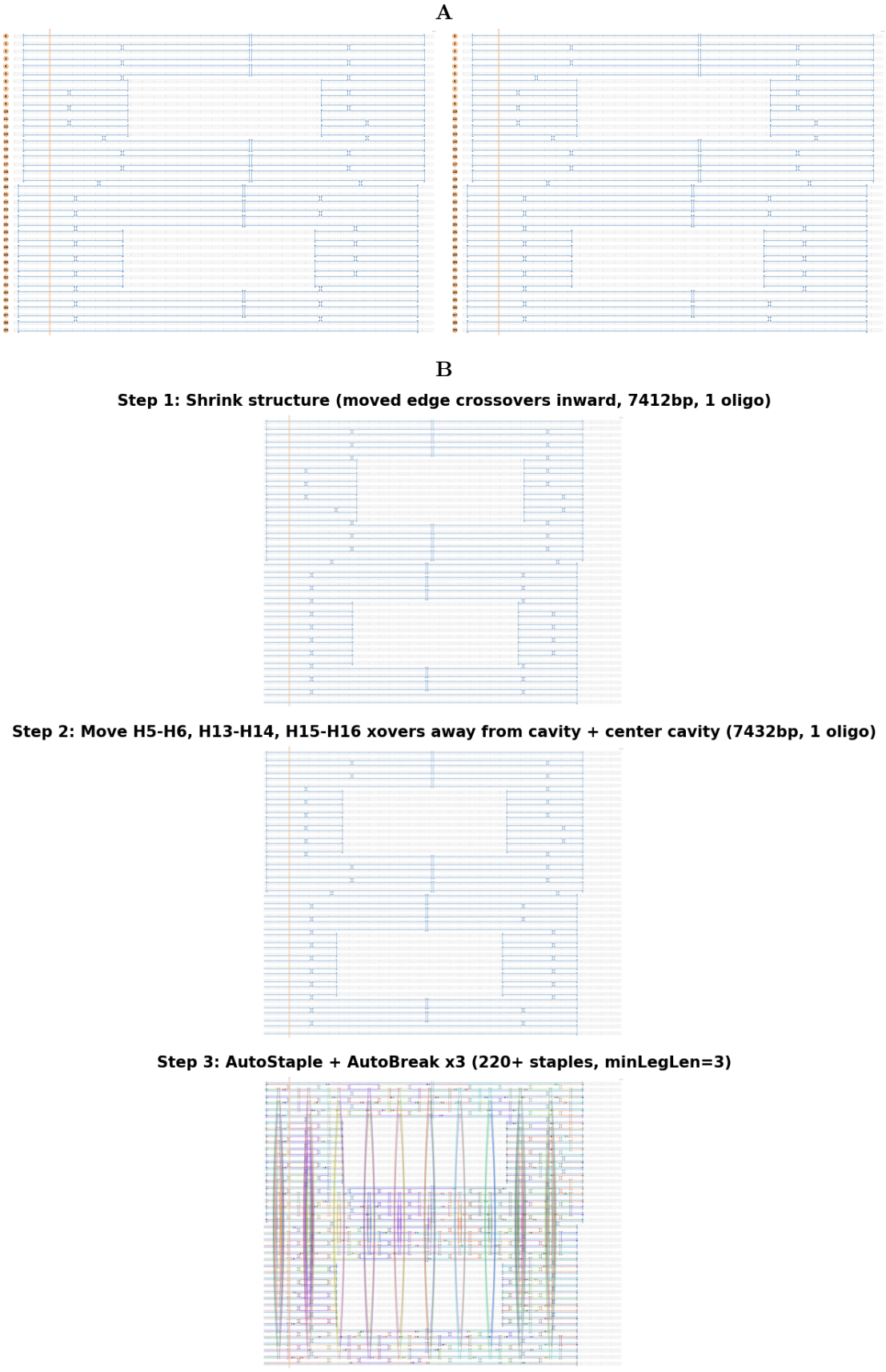

Figure 5: Targeted edits and end-to-end demonstrations. (A) Crossover repositioning: before (left, scaffold-only) and after (right). Inter-pair crossovers moved away from cavity boundaries, cavity centered. (B) Three-panel caDNAno2 path view sequence showing the 2-by-22 design after: shrink (left), crossover repositioning + cavity centering (center), autoStaple + autoBreak (right). Each edit executed correctly on the first attempt.

8. Agents as a Substrate for Shared Design Intelligence

The agent cannot design DNA origami independently. However, the failure modes documented in this work each encode domain knowledge with three properties:

1. **Invisible in source code.** Rules such as "LOW position = crossover IN, HIGH = crossover OUT" cannot be deduced from caDNAno's codebase. They require explicit transfer from an experienced designer.
2. **Retained after single correction.** Once corrected, the agent does not repeat that error class. The correction is encoded in verifier functions, lessons files, and pipeline logic that persist across sessions.
3. **Distributable.** The verifier (`cadnano_verifier.py`) checks all 10 failure modes automatically. The failure catalog (`failure_analysis.md`) documents each with symptom, root cause, fix, and verifier check. A new user or agent can apply these checks without repeating the original debugging.

The agent functions not as a designer but as a substrate for accumulating and distributing design knowledge (Figure 1, C). Each user-agent interaction produces reusable verification and pipeline tools in addition to the design itself.

The following artifacts (Table 1) are distributable to any DNA origami researcher working with caDNAno:

Artifact	**Purpose**	**Reusability**
`cadnano_verifier.py`	Pre-flight design validation	Run on any caDNAno JSON before committing to synthesis
`failure_analysis.md`	Documented failure modes	Read before starting AI-assisted design; avoid 10 known pitfalls
`cavity_variant_sweep.py`	Parametric cavity pipeline	Change gap size/width → design recomputes automatically
Template extension functions	Add columns to existing designs	Extend any 2-by-N template to 2-by-(N+2) with 1 scaffold oligo
Crossover move pattern	LOW=IN, HIGH=OUT template	Apply to any targeted crossover edit on any honeycomb design
Renumbering procedure	JSON helix renumbering	Essential for any template extension; ensures parity correctness

Table 1: Distributable artifacts from this work.

The pipeline from the user's template to verified design to oxDNA simulation is end-to-end: no manual caDNAno GUI interaction required. A researcher can describe a cavity design in terms of physical dimensions (nm × nm, scaffold length, padding) and receive a validated JSON, stapled design, and oxDNA files ready for submission.

Initial agent outputs (a flat sheet, a simple rectangle) demonstrated pipeline functionality but contained no transferable design knowledge. The transition to knowledge transfer occurred when the agent began generating verification tools from its own failure modes. The verifier was not designed top-down; it emerged from 10 specific debugging sessions, each producing a check that prevents the corresponding failure for any subsequent user.

References

(1) Majikes, J. M.; Liddle, J. A. DNA Origami Design: A How-to Tutorial. *Journal of Research of the National Institute of Standards and Technology* **2021**, *126*, 126001.

(2) Aksel, T.; Navarro, E. J.; Fong, N.; Douglas, S. M. Design Principles for Accurate Folding of DNA Origami. *Proceedings of the National Academy of Sciences* **2024**, *121* (48).

(3) Douglas, S. M.; Marblestone, A. H.; Teerapittayanon, S.; Vazquez, A.; Church, G. M.; Shih, W. M. Rapid Prototyping of 3d DNA-Origami Shapes with Cadnano. *Nucleic Acids Research* **2009**, *37* (15), 5001–5006.

(4) Fu, D.; Narayanan, R. P.; Prasad, A.; Zhang, F.; Williams, D.; Schreck, J. S.; Yan, H.; Reif, J. Automated Design of 3d DNA Origami with Non-Rasterized 2d Curvature. *Science Advances* **2022**, *8* (51).

(5) Veneziano, R.; Ratanalert, S.; Zhang, K.; Zhang, F.; Yan, H.; Chiu, W.; Bathe, M. Designer Nanoscale DNA Assemblies Programmed from the Top down. *Science* **2016**, *352* (6293), 1534–1534.

(6) Benson, E.; Mohammed, A.; Gardell, J.; Masich, S.; Czeizler, E.; Orponen, P.; Högberg, B. DNA Rendering of Polyhedral Meshes at the Nanoscale. *Nature* **2015**, *523* (7561), 441–444.

(7) Jun, H.; Zhang, F.; Shepherd, T.; Ratanalert, S.; Qi, X.; Yan, H.; Bathe, M. Autonomously Designed Free-Form 2d DNA Origami. *Science Advances* **2019**, *5* (1).

(8) Jun, H.; Wang, X.; Bricker, W. P.; Bathe, M. Automated Sequence Design of 2d Wireframe DNA Origami with Honeycomb Edges. *Nature Communications* **2019**, *10* (1).

(9) Levy, N.; Schabanel, N. Ensnano: A 3d Modeling Software for DNA Nanostructures. *DNA 27, Leibniz International Proceedings in Informatics* **2021**, *205*, 5:1–5:23.

(10) Rothemund, P. W. K. Folding DNA to Create Nanoscale Shapes and Patterns. *Nature* **2006**, *440* (7082), 297–302.

(11) Douglas, S. M.; Dietz, H.; Liedl, T.; Högberg, B.; Graf, F.; Shih, W. M. Self-Assembly of DNA into Nanoscale Three-Dimensional Shapes. *Nature* **2009**, *459* (7245), 414–418.

(12) Ke, Y.; Douglas, S. M.; Liu, M.; Sharma, J.; Cheng, A.; Leung, A.; Liu, Y.; Shih, W. M.; Yan, H. Multilayer DNA Origami Packed on a Square Lattice. *Journal of the American Chemical Society* **2009**, *131* (43), 15903–15908.

(13) Dietz, H.; Douglas, S. M.; Shih, W. M. Folding DNA into Twisted and Curved Nanoscale Shapes. *Science* **2009**, *325* (5941), 725–730.

(14) Ke, Y.; Voigt, N. V.; Gothelf, K. V.; Shih, W. M. Multilayer DNA Origami Packed on Hexagonal and Hybrid Lattices. *Journal of the American Chemical Society* **2012**, *134* (3), 1770–1774.

(15) He, J.; Treude, C.; Lo, D. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, And the Road Ahead. *ACM Transactions on Software Engineering and Methodology* **2025**, *34* (5), 1–30.

(16) Dong, Y.; Jiang, X.; Qian, J.; Wang, T.; Zhang, K.; Jin, Z.; Li, G. A Survey on Code Generation with LLM-Based Agents. *arXiv preprint arXiv:2508.00083* **2025**.

(17) Suma, A.; Poppleton, E.; Matthies, M.; Šulc, P.; Romano, F.; Louis, A. A.; Doye, J. P. K.; Micheletti, C.; Rovigatti, L. Tacoxdna: A User-Friendly Web Server for Simulations of Complex DNA Structures, From Single Strands to Origami. *Journal of Computational Chemistry* **2019**, *40* (29), 2586–2595.

(18) Ouldridge, T. E.; Louis, A. A.; Doye, J. P. K. Structural, Mechanical, And Thermodynamic Properties of a Coarse-Grained DNA Model. *The Journal of Chemical Physics* **2011**, *134* (8), 85101.

(19) Šulc, P.; Romano, F.; Ouldridge, T. E.; Rovigatti, L.; Doye, J. P. K.; Louis, A. A. Sequence-Dependent Thermodynamics of a Coarse-Grained DNA Model. *The Journal of Chemical Physics* **2012**, *137* (13), 135101.

(20) Snodin, B. E. K.; Randisi, F.; Mosayebi, M.; Šulc, P.; Schreck, J. S.; Romano, F.; Ouldridge, T. E.; Tsukanov, R.; Nir, E.; Louis, A. A.; Doye, J. P. K. Introducing Improved Structural Properties and Salt Dependence into a Coarse-Grained Model of DNA. *The Journal of Chemical Physics* **2015**, *142* (23), 234901.

(21) Rovigatti, L.; Šulc, P.; Reguly, I. Z.; Romano, F. A Comparison between Parallelization Approaches in Molecular Dynamics Simulations on Gpus. *Journal of Computational Chemistry* **2015**, *36* (1).

(22) Guo, D.; Yang, D.; Zhang, H.; Song, J.; Wang, P.; Zhu, Q.; Xu, R.; Zhang, R.; Ma, S. Deepseek-R1: Incentivizing Reasoning Capability in Llms via Reinforcement Learning. *Nature* **2025**, *645* (8081), 633–638.

(23) Wang, X.; Chen, Y.; Yuan, L.; Zhang, Y.; Li, Y.; Peng, H.; Ji, H. Executable Code Actions Elicit Better LLM Agents. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*; 2024.